

Vertex AI ile Metin Üretimi ve Parametre Optimizasyonu

text-bison Modeli ve Hiperparametre Kontrolü Teknik Çalışma Raporu

Büyük dil modelleri (LLM), kendilerine sunulan girdileri (prompt) olarak bir sonraki adımda gelmesi en olası kelime veya karakter gruplarını (token) tahmin eden güçlü üretken sistemlerdir. Bu çalışma raporu, Google Cloud'un Vertex AI platformunda yer alan **text-bison** modelini kullanarak metin üretme adımlarını, prompt mühendisliği taktiklerini ve çıktı kalitesini doğrudan etkileyen hiperparametrelerin (Sıcaklık, Top-K, Top-P) çalışma mekanizmalarını detaylı kod örnekleri ve günlük hayattan analogilerle ele almaktadır.

1. Model Sınıflandırması: Text-Bison vs. Chat-Bison

Vertex AI ekosisteminde farklı görevler için özelleştirilmiş model varyantları bulunur. **TextBison** modeli; duygu analizi, metin sınıflandırma, özetleme ve bilgi çıkarma gibi *tek bir API isteği ve yanıtı (single-turn)* ile tamamlanabilen görevler için ideal hale getirilmiştir. Öte yandan, eğer arka arkaya süren, geçmişini hatırlaması gereken karşılıklı konuşmalar (multi-turn) içeren bir sohbet arayüzü inşa etmek istiyorsanız, bunun için özel olarak eğitilmiş **ChatBison** modelini tercih etmeniz gerekir.

2. Vertex AI SDK Kurulumu ve Kimlik Doğrulama

Vertex AI servislerini Python ortamında kullanabilmek için öncelikle ilgili kütüphanelerin içe aktarılması, Google Cloud kimlik bilgilerinin doğrulanması ve SDK'nin bölge (region) bilgisiyle ilklendirilmesi gerekir. Aşağıdaki kod bloğunda gerekli kurulum ve ilklendirme adımları gösterilmiştir:

```
import vertexai from vertexai.language_models import TextGenerationModel # Google Cloud projesini ve bölgesini tanımlama PROJECT_ID = "gcp-projeniz" REGION = "us-central1" # SDK'in ilklendirilmesi ve kimlik doğrulama adımları vertexai.init(project=PROJECT_ID, location=REGION) # Metin üretimi için TextBison modelinin (001 versiyonu) yüklenmesi model = TextGenerationModel.from_pretrained("text-bison@001")
```

3. İlk predict() Çağırısı ve Açık Uçlu Üretim

Model yüklendikten sonra, predict() fonksiyonu aracılığıyla hazırladığımız promptu modele iletebiliriz. Bu ilk örnekte, bir lise öğrencisine programlama aktiviteleri önermesini isteyen açık uçlu ve yaratıcı bir istem (prompt) kullanılmıştır:

```
# Açık uçlu yaratıcı bir prompt tanımlama prompt = """Ben bir lise öğrencisiyim. Yeteneklerimi geliştirmek için bana bir programlama aktivitesi önerir misin?""" # Modeli çağırarak yanıtı alma ve yazdırma response = model.predict(prompt) print(response.text)
```

Model Çıktısı Analizi: Model bu durumda oldukça esnek, açık uçlu ve yaratıcı bir yanıt üreterek öğrencinin ilgilendiği bir problemi çözecek bir yazılım geliştirmesini, çevrimiçi kurslara katılmasını önermektedir. Bu esneklik beyin fırtınası için harikadır ancak çıktı değişkenliği yüksektir.

Prompt Mühendisliği ve Bilgi Çıkarımı

1. İstem Mühendisliği ile Davranış Sınırlandırma (Classification)

İstediğimiz amaca ulaşmak için model girdilerini stratejik olarak tasarlama sanatına **Prompt Mühendisliği (Prompt Engineering)** adı verilir. Eğer modelin çok geniş ve değişken yanıtlar vermesini istemiyor, daha kısıtlı ve belirli sınırlar içinde kalmasını istiyorsak açık uçlu bir soruyu sınıflandırma görevine dönüştürebiliriz. Bu, çıktının değişkenliğini (variability) ciddi oranda azaltır.

```
# Sınıflandırılmış ve seçenekli prompt tasarımı prompt = """Ben bir lise öğrencisiyim. Aşağıdaki aktivitelerden hangisini yapmamı önerirsin ve neden? A) Python Öğrenmek B) JavaScript Öğrenmek C) Fortran Öğrenmek""" response = model.predict(prompt) print(response.text)
```

Sonuç: Model bu durumda seçenekler dışına çıkmayarak genellikle A seçeneğini (Python Öğrenmek) seçecek ve bunu yapılandırılmış bir mantıkla nedenleriyle birlikte sunacaktır. İstem tasarımıdaki ufak değişiklikler, LLM'in davranışını tamamen değiştirebilir.

2. Yapılandırılmamış Metinden Bilgi Çıkarımı (Information Extraction)

Büyük dil modellerinin en güçlü yeteneklerinden biri de düz metin formatında verilmiş yapılandırılmamış verileri analiz ederek belirli şablonlara göre yapılandırılmış formata (tablo, JSON vb.) dönüştürebilmesidir. Örneğin, hayali bir yaban hayatı biyoloğu filminin oyuncu kadrosu ve karakter analizini içeren düz metinden karakterlerin adlarını, mesleklerini ve onları canlandıran aktörleri çekip çıkarabiliriz.

```
# Yapılandırılmamış film sinopsis metni ve bilgi çıkarma talimatı synopsis_text = """ Yaban hayatı biyoloğu Dr. Elena Vance (Alice Green tarafından canlandırılıyor), arazi rehberi Marcus Brody (John Smith canlandırıyor) ile birlikte Amazon ormanlarında yeni bir tür keşfeder. Proje laboratuvar şefi Dr. Sarah Jenkins (Emma Stone canlandırıyor) ise merkezden destek sağlar. """ prompt = f"""Yukarıdaki film sinopsisinden karakterleri, yaptıkları işleri ve onları canlandıran aktörlerin isimlerini çekip çıkar. Bu bilgileri bana bir tablo formatında (Markdown) sun: Metin: {synopsis_text} """ response = model.predict(prompt) print(response.text)
```

Model Tarafından Üretilen ve Çıkarılan Tablo Örneği:

Karakter (Character)	Görev / Meslek (Job)	Aktör (Actor)
Dr. Elena Vance	Yaban Hayatı Biyoloğu	Alice Green
Marcus Brody	Arazi Rehberi	John Smith
Dr. Sarah Jenkins	Laboratuvar Şefi	Emma Stone

Bu çıktı modeli, veri analistlerinin veya içerik yöneticilerinin düz metinler üzerinde saatlerce uğraşarak yapacağı veri sınıflandırma ve temizleme süreçlerini saniyeler içinde otomatize etmelerini sağlar.

Hiperparametre Kontrolü ve Sıcaklık (Temperature)

1. Dil Modellerinin Arka Plan Çalışması ve Token Tahmini

LLM'ler temelde kelimeleri değil, **token** adı verilen temel dil birimlerini işler. Bir prompt girdikten sonra model, bir sonraki adımda gelebilecek tüm olası token'lar için bir **olasılık dizisi (array of probabilities)** hesaplar. Örneğin, 'Bahçe çok güzel...' cümlesinden sonra gelebilecek kelimeler: Çiçeklerle (%50), Ağaçlarla (%23), Taşlarla (%5) gibi olasılıklarla sıralanır. Bu olasılık havuzundan bir kelime seçme stratejisine **kod çözme stratejisi (decoding strategy)** denir.

Açgözlü Kod Çözme (Greedy Decoding): Her adımda sadece en yüksek olasılığa sahip token'ı seçer. Bu yaklaşım tutarlıdır ancak yaratıcı olmayan, tekrarlayan ve sıkıcı çıktılara yol açabilir.

Örnekleme (Sampling): Olasılık dağılımından rastgele seçimler yapar. Bu sayede daha nadir ve ilginç kelimeler seçilebilir, ancak rastgelelik kontrol edilmezse anlamsız ifadeler ortaya çıkabilir.

2. Sıcaklık (Temperature) Parametresi

Sıcaklık, olasılık dağılımının ne kadar 'keskin' veya 'düz' olacağını belirleyerek rastgelelik derecesini kontrol eder. Teknik olarak, modelin ham çıktı skorları (logits) Softmax fonksiyonuna gönderilmeden önce sıcaklık değerine (theta) bölünür:

Softmax Formülü: $P(z_i) = \exp(z_i / \theta) / \sum(\exp(z_j / \theta))$

Burada **theta** (sıcaklık) düştükçe en yüksek olasılıklı token'ın seçilme payı domine eder. Sıcaklık 0 olduğunda sistem tamamen deterministik (belirlenimci) hale gelir; aynı prompt her çalıştırıldığında birebir aynı kelimeyi seçer.

Günlük Hayat Benzetmesi (Mutfaktaki Şef): Sıcaklık parametresini bir şefe benzetebiliriz. **Sıcaklık = 0** olduğunda şef, geleneksel bir İtalyan pizzası tarifine milimetrik olarak sadık kalır ve her seferinde kusursuz ama tamamen aynı pizzayı yapar. **Sıcaklık = 1** olduğunda ise şef mutfakta tamamen özgürdür; ananas, çikolata veya avokado gibi sıra dışı malzemeleri pizzaya ekleyebilir. Sonuç çok yaratıcı veya bazen yenmeyecek kadar tuhaf olabilir!

3. Sıcaklık Parametresinin Kod Üzerindeki Etkisi

Vertex AI'da sıcaklık değeri **0 ile 1** arasında belirlenebilir. Aşağıdaki kod örneğinde yarım bırakılmış bir cümle için 0 ve 1 sıcaklıklarında nasıl tamamlandığı karşılaştırılmıştır:

```
prompt = ""Cümleyi tamamla: Resim çerçevesini hazırlarken, alet çantamdan almak için uzandığım alet benim..."" # Deterministik tamamlama (Sıcaklık = 0) response_det = model.predict(prompt, temperature=0.0) print("Deterministik Çıktı:", response_det.text) # Çıktı her zaman: "...çekiçtir." (En olası araç) # Yaratıcı/Rastgele tamamlama (Sıcaklık = 1.0) response_creative = model.predict(prompt, temperature=1.0) print("Yaratıcı Çıktı:", response_creative.text) # Çıktı değişken olabilir: "...testeredir.", "...zımpara kağıdıdır." vb.
```

Gelişmiş Filtreleme: Top-K ve Top-P

1. Top-K Filtreleme (Sabit Seçim Havuzu)

Top-K, modelin olasılık listesindeki en yüksek puana sahip ilk K adet token'ı seçip geri kalanını tamamen elendiği bir stratejidir. Örneğin, $K=2$ olarak ayarlanırsa, model sadece en yüksek olasılığa sahip 2 kelimeyi (Örn: çiçekler ve ağaçlar) değerlendirmeye alır. Top-K'nın en büyük kısıtı, olasılık dağılımının şekline adapte olamamasıdır. Bir kelime çok baskın olsa bile arkasından gelen düşük olasılıklı kelimeler havuzda kalmaya devam eder. Vertex AI'da varsayılan değeri 40 olup, 1 ile 40 arasında ayarlanabilir.

2. Top-P Filtreleme (Dinamik/Nucleus Seçim Havuzu)

Top-K'nın kısıtlamalarını çözmek için geliştirilen **Top-P (Nucleus Sampling)** yöntemi, token'ları kümülatif olasılık değerlerine göre filtreler. En yüksek olasılıklı kelimedenden başlanarak, olasılıklar toplanır ve bu toplam belirlenen P eşik değerini aşana kadar kelimeler havuza eklenir. Örneğin, $P=0.75$ ise, çiçekler (%50) + ağaçlar (%23) + çimen (%5) = %78 toplamına ulaşıldığı an filtreleme durur ve sadece bu 3 kelimedenden örnekleme yapılır. Bu sayede seçim havuzunun boyutu olasılıkların dağılımına göre dinamik olarak genişler veya daralır. Varsayılan değeri 0.95 olan bu parametre, 0 ile 1 arasında değer alır.

3. Parametrelerin Birleşik Çalışma Sırası (Combined Execution Order)

Modele aynı anda hem Temperature, hem Top-K, hem de Top-P değerleri girildiğinde, sistem bu filtreleri rastgele değil, belirli bir sıra ve hiyerarşiyle uygular. Bu akış sırası şu şekildedir:

- 1. Aşama (Top-K Filtreleme):** Tüm olası kelimeler arasından en yüksek olasılıklı ilk K kelime seçilir, kalanlar elenir.
- 2. Aşama (Top-P Filtreleme):** Kalan K kelime içerisinde kümülatif olasılığı P değerini aşan en dar küme seçilir.
- 3. Aşama (Sıcaklık Örnekleme):** Son kalan dinamik kelime havuzunun olasılıkları Temperature değerine göre esnetilir/düzleştirilir ve nihai kelime seçilir.

4. Kapsamlı Uygulama ve Kombinasyon Kod Örneği

Aşağıdaki örnekte tüm bu parametreler bir arada kullanılarak yaratıcı, belirli sınırlar içinde tutarlı ve eğlenceli bir pazarlama reklam metni oluşturulmuştur. Sıra dışı nesnelerin (mavi fil ve avokado) bir araya getirilmesi için sıcaklık yüksek (0.9), fakat kelime havuzunun aşırı saçmalamaması için Top-P (0.2) ve Top-K (20) daha kısıtlayıcı tutulmuştur:

```
# Sıra dışı öğeler barındıran reklam promptu
prompt = """Mavi filler ve avokadolar içeren mont ve ceketler için yaratıcı ve dikkat çekici bir reklam yazısı hazırla."""
# Parametrelerin bir arada verilerek modelin çağırılması
response = model.predict(prompt, temperature=0.9, # Yaratıcılık yüksek tutuldu
top_p=0.2, # Dinamik olasılık havuzu daraltıldı (tutarlılık için)
top_k=20 # Başlangıç kelime kümesi yarı yarıya sınırlandırıldı )
print(response.text)
```

Sonuç Özet Tablosu:

Görev Tipi	Önerilen Parametre Ayarları	Kullanım Amacı
Sınıflandırma / Bilgi Çıkarma	Sıcaklık = 0.0, Top-P = Düşük	Tutarlı, tahmin edilebilir ve net çıktılar almak.
Yaratıcı Yazarlık / Beyin Fırtınası	Sıcaklık = 0.7 - 1.0, Top-P = Yüksek	Farklı, orijinal ve çeşitli kelime kombinasyonları.
Dengeli Yanıtlar (Soru-Cevap)	Sıcaklık = 0.2 - 0.5, Top-P = Varsayılan (0.95)	Hem doğru bilgiye sadık kalıp hem akıcı dil kullanmak.